



CMSC 472/672 Computer Vision

Homework 1. 70 points

Due: Sept 18, 2026. 23:59 Eastern

Answer all problems according to the instructions provided.

Submit one .zip file named `LastName_FirstName_hw1.zip` containing a single pdf named `hw1.pdf` with the answers to both the written and coding sections. The pdf **must** contain typeset or handwritten answers for the written problems and relevant code snippets, outputs and results, and other requested information for the programming tasks. Put all other files (code, data, outputs, etc.) in a single directory named “code”

Not following the instructions will result in a deduction of 10 points.

1 Point Processing (12)

For this section, we will use the following notation:

- the pixel intensity of image I at coordinates (x, y) is denoted by $I[x, y]$
- a point-wise image processing operation is represented by the function $f(x, y)$
- the output of a point processing operation is denoted by the resulting image $\hat{I}$

Using this notation, write the equations for point processing operations described below:

1. Setting all pixels with intensity less than 50 to 0 (2)
2. Adding pixel-wise Gaussian noise with mean μ and standard deviation σ to the image (2)
3. Gradual darkening an image from left to right (2)
4. Reducing the range of image intensities from $[0, 255]$ to $[50, 150]$ (2)
5. “Quantizing” the image to n fixed values, i.e. 256 possible pixel intensities between $[0, 255]$, we will only use n possible values between $[0, 255]$. What point processing equation would you use to achieve this? Write a general equation for any n . (*Hint: think about what you would do for $n = 2, 3, 4$ and then come up with a general formula*) (4)

2 Lenses (18)

1. Draw a ray diagram showing an object, a convex lens, and the image plane. Show the rays if the object is at infinity. Show the focal length of the lens in the ray diagram. You can make these drawings either by hand or with your computer (eg. Powerpoint, Paint, or any other diagram tools). (5)
2. You have a pinhole camera. Abraham Lincoln, a person with height 200cm , goes and stands 1m in front of the camera. If Lincoln demands that the image should have $20\times$ magnification, how far should the image plane be from the pinhole? (5)

3. You now have a new camera, with a lens. Lincoln demands that you must photograph his hat, which has a height and width of 25cm . You have a lens with focal length 35mm and the image sensor placed at the focus. The image sensor width and height is 50mm .
 - (a) What is the minimum distance between Lincoln's hat and the camera for the entire hat to fit within the photo? (4)
 - (b) What is the distance if Lincoln demands that the hat should only occupy quarter of the area of the image. (4)

3 Basic Operations in OpenCV (20)

This problem is designed to help you learn how to implement basic image processing operations in Python. We will use Python along with OpenCV and additional helper libraries to do most of the image processing required in homework assignments. You will not need anything other than your laptop or desktop for this homework assignment. No GPUs are necessary.

Task 1 *Setting up your coding environment* 0 points

1. Install Anaconda Python <https://docs.anaconda.com/anaconda/install/> and make sure you follow the instructions specific to your operating system. To test if your Python is working, create a file `helloworld.py` with the following line:

```
1 print("Hello World")
```

and run it using the following command in a terminal shell / command prompt:

```
1 python helloworld.py
```

2. Install OpenCV via Anaconda using the following command:

```
1 conda install -c conda-forge opencv
```

3. Install Matplotlib (a useful library for visualizing graphical outputs and for making plots)

```
1 conda install matplotlib
```

4. Make sure your libraries work by importing them. Create a file named `testimport.py` with the following lines:

```
1 import cv2 # this imports OpenCV
2 import numpy as np # this imports numpy
3 import matplotlib.pyplot as plt
```

If you get no errors at this point, you have successfully set up your environment. **NOTE:** if you are stuck on any step, Google is your friend! Just search something like “How do I install X on a iMac/Windows/Linux/...?” or you can even copy your error message and google it.

5. Note for documenting code: for every section, you should block it off with a set of comments. This makes it easy to read and legible. For example:

```

1      import cv2 # this imports OpenCV
2      import numpy as np # this imports numpy
3      import matplotlib.pyplot as plt
4
5      ##### Section 2.X - <insert description> #####
6      # load image
7      <your code here>
8      # display image
9      <your code here>
10     ...
11     ..
12     #####
13

```

Task 2 *Loading and Displaying Images*

5 points

1. Read in the image `elephant.jpeg` that is provided in the homework assignment. Useful functions: `cv2.imread`, `cv2.imshow`, `cv2.waitKey`, `cv2.destroyAllWindows`. The latter makes sure that pressing a button will close all windows (otherwise the code will remain stuck on displaying the image).
2. Display the image using matplotlib. Useful functions: `plt.imshow`, `plt.show`. Write the image to a file named `elephant_opencv.png` by using the command `cv2.imwrite`. Do you notice anything weird about the colors?
3. OpenCV stores images in BGR format, but Matplotlib expects RGB! This is the reason why the image `elephant_opencv.png` had inverted colors. Let's fix this. To load the elephant image with correct colors for Matplotlib, use `cv2.cvtColor` to convert the loaded image to a RGB space, then plot with Matplotlib.

Write this image to a file named `elephant_matplotlib.png`.

Task 3 *Basic Image Processing Operations*

15 points

1. Convert the image to grayscale. Read (or convert the existing image) as grayscale, plot it (hint: look up the `cmap` option) and write the output to `elephant_gray.png`.
2. **Cropping:** Read in the elephant image in color. Crop out the small elephant by figuring out the x,y coordinates needed. Note that the first coordinates in Python correspond to image rows, and the second coordinates correspond to image columns, and the last set of coordinates are the color channels. Display the image of the small elephant (correctly in RGB) using Matplotlib, and then write out the image as `elephant_baby.png`.
3. **Resizing:**
 - (a) Read in the image in color again, and convert it to RGB space. Downsample the image by 10× in width and height. Use the `cv2.resize` command. If you have problems with this command, look at examples online and use the keyword “None” for the optional parameters. Display the image, and write out the image as `elephant_10xdown.png`.

- (b) Using the `resize` command, upsample the same downsampled image from part b by $10\times$ back to its original resolution. This time, try two different interpolation methods: nearest neighbor and bicubic. Write out both images to files as `elephant_10xup_method.png` where `method` is the method you used for interpolation.
- (c) Calculate the absolute difference between the ground truth image and the two upsampled images with the two methods (find the appropriate OpenCV command, should be one line of code for each image). Write out the difference images for both methods. **Sum all the pixels in the difference image for the two methods, and report the number. Which method caused less error in upsampling?**

4 Edge Detection and Image Blurring by Convolution (20)

1. Write a function using Python and OpenCV to implement edge detection using 2D convolution. (Hint: look up `cv2.filter2D`. Show the filter that you used for this operation, written as a matrix. (4)
2. Repeat step 1, but with a different edge detection filter (for example, you can use a larger filter size, or different filter values) (2)
3. Similarly, implement image blurring (also known as smoothing) using the same OpenCV function you used for edge detection. Show the filter that you used for this operation, written as a matrix. (4)
4. Repeat step 3, but with a different blurring filter (for example, you can use a larger filter size, or Box/Gaussian/other types of blurring filters). (2)
5. Display the original elephant image, the two edge detection outputs, and the two blurring outputs, side by side. (2)
6. **Convolution from scratch:** Implement convolution without using any OpenCV or Numpy functions, but purely with Python. Apply it to the image and show the output with one edge detection filter and one blurring filter of your choice. (6)

5 Extra Credit: Images as Matrices (10)

In addition to your own thoughts about these question, you are allowed to use sources such as research articles, as long as you understand and explain ideas from those sources. If you do so, please cite these sources.

1. In class, we studied how to represent images as a matrix. Is every matrix an image? You can provide answers for or against this argument or argue on both sides. (5)
2. I have a camera that produces color images with 1000 rows and 1000 columns. I store pixel intensities as integers in the range $[0, 255]$. How many bytes of storage will I need to store this image? If I store this image in my computer as `.jpeg`, it will occupy approximately 500 kilobytes. Is this different from your answer? Why? (5)